

Training Piscine JavaScript for backend – 1

Day02 – Objects & Data Structures

Summary: Today, you will discover objects: literals, property access, key/value iteration, destructuring, the spread operator, and JSON serialization.

Version: 1.0

Contents

1	General rules	2
2	Specific instructions of the day	3
3	Exercise 00	4
4	Exercise 01	5
5	Exercise 02	6
6	Exercise 03	7
7	Exercise 04	8
8	Exercise 05	9
9	Exercise 06	10
10	Exercise 07	11
11	Submission and peer-evaluation	12

Chapter 1

General rules

- You have to render your modules from a computer in the cluster, either using a virtual machine or the computer directly if the tools are available.
- Your functions should not crash unexpectedly. If this happens, your project will be considered non-functional and will receive a 0 during the evaluation.
- We encourage you to write test programs for your project, even though this work **won't have to be submitted and won't be graded**.
- Submit your work to your assigned Git repository. Only the work in the Git repository will be graded.
- You must use **Node.js (v18+)**.
- Your code must follow standard **ESLint** rules (2-space indentation, no global leakage).
- There is no global variable.

Chapter 2

Specific instructions of the day

- No code in the global scope outside of function definitions.
- Each file must contain **only** the requested function, plus its `module.exports`.
- Any exception not caught will invalidate the exercise.
- You can use any built-in function unless explicitly prohibited.
- All your functions must be documented using JSDoc comments where requested.

Chapter 3

Exercise 00

Exercise 00: Build and access
Turn-in directory: ex00/
Files to turn in: ft_build_access.js
Allowed functions: object literals, dot notation, bracket notation, console.log()

Write a function named `ft_build_access` that takes a **key** (string) and a **value** (string) as parameters. Inside the function, build an **object literal** representing a product with 3 fixed properties: **name** ("Keyboard"), **price** (49), **inStock** (**true**). Then:

- log the **name** property using **dot notation**
- log the **price** property using **bracket notation**
- add a new property to the object using the given **key** and **value** (bracket notation is required here, since the property name is not known in advance), then log the whole object

Function prototype:

```
/**
 * @param {string} key
 * @param {string} value
 * @returns {void}
 */
function ft_build_access(key, value) {
    // your code here
}
```

```
ft_build_access("color", "black");
Name:   Keyboard
Price:  49
{ name: 'Keyboard', price: 49, inStock: true, color: 'black' }
```

Reminder: dot notation (`obj.prop`) only works with a property name known at write time. Bracket notation (`obj["prop"]` or `obj[variable]`) also accepts a property name stored in a variable.

Chapter 4

Exercise 01

Exercise 01: List the keys
Turn-in directory: ex01/
Files to turn in: ft_list_keys.js
Allowed functions: Object.keys(), console.log()

Write a function named `ft_list_keys` that takes an object as parameter and logs the number of properties it has, followed by the array of its property names, using `Object.keys()`.

Function prototype:

```
/**
 * @param {Object} obj
 * @returns {void}
 */
function ft_list_keys(obj) {
    // your code here
}
```

```
ft_list_keys({ title: "1984", author: "Orwell", year: 1949 });
Property count: 3
Keys: [ 'title', 'author', 'year' ]
```

Chapter 5

Exercise 02

Exercise 02: List the values
Turn-in directory: ex02/
Files to turn in: ft_list_values.js
Allowed functions: Object.values(), console.log()

Write a function named `ft_list_values` that takes an object as parameter and logs the array of its property values, using `Object.values()`.

Function prototype:

```
/**
 * @param {Object} obj
 * @returns {void}
 */
function ft_list_values(obj) {
    // your code here
}
```

```
ft_list_values({ title: "1984", author: "Orwell", year: 1949 });
Values: [ '1984', 'Orwell', 1949 ]
```

Reminder: `Object.keys()` and `Object.values()` both return **arrays**, always in the same property order, so `Object.keys(obj)[i]` and `Object.values(obj)[i]` refer to the same property.

Chapter 6

Exercise 03

Exercise 03: Entries report
Turn-in directory: ex03/
Files to turn in: ft_entries_report.js
Allowed functions: Object.entries(), for...of, template literals, console.log()

Write a function named `ft_entries_report` that takes an object as parameter, then uses `Object.entries()` combined with a `for...of` loop to log every property as `"key: value"`.

Function prototype:

```
/**
 * @param {Object} obj
 * @returns {void}
 */
function ft_entries_report(obj) {
  // your code here
}
```

```
ft_entries_report({ title: "1984", author: "Orwell", year: 1949 });
title: 1984
author: Orwell
year: 1949
```

Reminder: `Object.entries(obj)` returns an array of `[key, value]` pairs. Each pair is itself a 2-element array, which you can destructure directly in the loop: `for (const [k, v] of Object.entries(obj))`.

Chapter 7

Exercise 04

Exercise 04: Destructure me
Turn-in directory: ex04/
Files to turn in: ft_destructure_me.js
Allowed functions: object destructuring, template literals, console.log()

Write a function named `ft_destructure_me` that takes an object representing a user (with properties `name`, `age`, `city`) as parameter. Inside the function, **destructure** the 3 properties into individual variables in a single line, then log a sentence built with a template literal.

Function prototype:

```
/**
 * @param {Object} user
 * @returns {void}
 */
function ft_destructure_me(user) {
  // your code here
}
```

```
ft_destructure_me({ name: "Yassine", age: 25, city: "Rabat" });
Yassine is 25 years old and lives in Rabat.
```

Reminder: destructuring syntax: `const { name, age, city } = user;` creates 3 new variables at once, matched by property name.

Chapter 8

Exercise 05

Exercise 05: Merge profiles
Turn-in directory: ex05/
Files to turn in: ft_merge_profiles.js
Allowed functions: spread operator {...obj}, console.log()

Write a function named `ft_merge_profiles` that takes two objects, `defaults` and `overrides`, as parameters. Using the **spread operator**, build and log a new object that contains all the properties of `defaults`, with any matching property overwritten by the value from `overrides`. Neither input object should be modified.

Function prototype:

```
/**
 * @param {Object} defaults
 * @param {Object} overrides
 * @returns {void}
 */
function ft_merge_profiles(defaults, overrides) {
  // your code here
}
```

```
ft_merge_profiles(
{ theme: "light", notifications: true },
{ theme: "dark" }
);
{ theme: 'dark', notifications: true }
```

Warning: `{ ...defaults, ...overrides }` only performs a **shallow** copy/merge. Nested objects inside `defaults` or `overrides` are still shared by reference, not duplicated.

Chapter 9

Exercise 06

Exercise 06: To JSON
Turn-in directory: ex06/
Files to turn in: ft_to_json.js
Allowed functions: JSON.stringify(), typeof, console.log()

Write a function named `ft_to_json` that takes an object as parameter, converts it to a JSON string using `JSON.stringify()`, then logs both the resulting string and its `typeof`.

Function prototype:

```
/**
 * @param {Object} obj
 * @returns {void}
 */
function ft_to_json(obj) {
    // your code here
}
```

```
ft_to_json({ title: "1984", year: 1949 });
{"title":"1984","year":1949}
Type: string
```

Chapter 10

Exercise 07

Exercise 07: From JSON
Turn-in directory: ex07/
Files to turn in: ft_from_json.js
Allowed functions: JSON.parse(), typeof, dot notation, console.log()

Write a function named `ft_from_json` that takes a JSON string as parameter, converts it back into an object using `JSON.parse()`, logs its `typeof`, then logs one of its properties using dot notation.

Function prototype:

```
/**
 * @param {string} jsonString
 * @returns {void}
 */
function ft_from_json(jsonString) {
    // your code here
}
```

```
ft_from_json('{"title":"1984","year":1949}');
Type:   object
Title:  1984
```

Reminder: `JSON.stringify()` and `JSON.parse()` are opposite operations: `object` $\rightarrow$ `string`, and `string` $\rightarrow$ `object`. This round-trip is exactly what happens when data travels through a backend API.

Chapter 11

Submission and peer-evaluation

Turn in your assignment in your Git repository as usual. Only the work inside your repository will be evaluated during the defense.

The evaluation process will happen on the computer of the evaluated group.